CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 18: Evaluate Connection Protocols

Exam objective: Evaluate connection protocols for Claude-based integrations, including the Model Context Protocol (MCP), direct API integration, CLI-based integration, agent-to-agent (A2A) communication, choosing the appropriate protocol, understanding client/server responsibilities, tool discovery and execution, and authentication and permission boundaries.

Q1 **An enterprise wants to connect its Claude-based assistant to a dozen internal systems (CRM, ticketing, inventory, HR) in a way that lets multiple different client applications (a chat UI, a Slack bot, an internal developer tool) all reuse the same standardized tool integrations without re-implementing API logic for each client. Which protocol approach is best suited?**

A. Direct, bespoke API integration written separately inside each client application

B. **Model Context Protocol (MCP), where each system exposes an MCP server with standardized tool definitions that any MCP-compatible client can discover and call consistently**

C. A CLI tool that must be manually invoked by a human for every single request

D. Embedding all business logic directly into the system prompt as plain text instructions

CORRECT ANSWER B

WHY MCP is designed precisely for this reuse scenario: a standardized server exposes tools/resources once, and any compliant client (chat UI, Slack bot, internal tool) can discover and invoke them consistently, avoiding duplicated integration logic per client. Bespoke per-client API integration duplicates effort and is harder to maintain, a CLI requiring manual human invocation defeats the purpose of an automated assistant, and embedding business logic as prompt text provides no real execution capability or reusable interface.

Q2 **A fintech startup needs its Claude agent to call a single, well-documented internal payments API that only this one application will ever use, with no need for interoperability with other clients or tools. What is a reasonable protocol choice given this narrow scope?**

A **Direct API integration, since the overhead of standing up a full MCP server and tool-discovery layer may not be justified for a single-purpose, single-client integration**

B Agent-to-agent (A2A) protocol, since any integration between software components should default to A2A regardless of scope

C A CLI-based integration, since CLI tools are always more secure than APIs

D MCP is mandatory for all Claude integrations regardless of use case

CORRECT ANSWER **A**

WHY Direct API integration is a reasonable, simpler choice when there is a single client and no need for cross-client tool discovery/reuse, avoiding the added complexity of a general-purpose protocol layer that provides benefits (shared tool discovery, multiple clients) the use case does not need. A2A is designed for agent-to-agent coordination scenarios, not generic single-client API access, CLI tools are not inherently more secure than well-secured APIs, and MCP is not mandatory for every integration — protocol choice should match the actual integration topology and reuse needs.

Q3 **An architect is designing a system where a customer-support orchestrator agent needs to delegate a specialized sub-task (fraud risk scoring) to a separate, independently operated fraud-detection agent built by a different team, potentially even a different organization, where each agent maintains its own reasoning and state. Which protocol concept is most relevant?**

A MCP tool discovery, since fraud scoring is just another tool to be called

B **Agent-to-agent (A2A) communication, which is designed for coordination between autonomous agents that each have their own reasoning/state, rather than simple stateless tool invocation**

C CLI-based integration, since command-line tools are the standard way agents coordinate

D There is no meaningful distinction between this scenario and a simple API call; any protocol works identically

CORRECT ANSWER **B**

WHY A2A protocols are specifically designed for peer agent coordination where the called party is itself an autonomous agent with its own reasoning and potentially long-running task state, which differs from a straightforward stateless tool call that MCP typically models. Framing fraud scoring as "just a tool" undersells the complexity of coordinating with an independent reasoning agent, CLI integration is not the standard mechanism for agent coordination, and asserting no meaningful distinction ignores real architectural differences between simple tool invocation and multi-agent coordination.

Q4 **A healthcare software vendor is building an MCP server to expose clinical data tools to multiple hospital customers' Claude deployments. Which statement correctly describes client/server responsibilities in this MCP architecture?**

A The MCP client is responsible for enforcing all data access permissions, while the MCP server merely returns whatever the client requests

B **The MCP server is responsible for defining available tools/resources and enforcing data-level authorization for the data it exposes, while the MCP client (the Claude-integrated application) is responsible for initiating requests, managing the conversation, and passing the appropriate authenticated context to the server**

C Both client and server share identical responsibilities with no meaningful division of labor

D The MCP protocol itself has no concept of client or server roles; all components are peers

CORRECT ANSWER **B**

WHY In MCP's architecture, the server owns and exposes the tools/resources (and must enforce authorization for the data it controls), while the client drives the conversation and must supply proper authenticated context when calling the server — this division of responsibility is fundamental to the protocol's design, especially critical for a healthcare vendor serving multiple hospital tenants. Making the client solely responsible for all access enforcement is backwards and dangerous (the server, as the data owner, must enforce its own access rules), claiming identical responsibilities ignores the clear architectural split, and denying any client/server distinction misrepresents MCP's actual client-server model.

Q5 **A retail enterprise's developer platform team wants engineers to be able to quickly test and debug a new Claude-powered internal tool integration from their local terminal during development, before wiring it into the production chat UI. Which integration approach best fits this specific development-time need?**

A **A CLI-based integration, allowing developers to invoke and inspect tool calls interactively from the command line during development and debugging**

B Full production-grade MCP server deployment with load balancing, required before any local testing can begin

C Agent-to-agent protocol negotiation between two production agents

D Embedding test credentials permanently into the production chat UI's client-side code

CORRECT ANSWER **A**

WHY CLI-based integration is well suited to developer-facing, interactive testing and debugging scenarios, letting engineers quickly invoke and inspect tool behavior from a terminal without needing full production infrastructure. Requiring a fully deployed, load-balanced MCP server just to test locally adds unnecessary overhead for this stage, A2A protocol negotiation is irrelevant to a single developer testing a tool locally, and embedding test credentials in production client-side code is a serious security anti-pattern unrelated to the actual need.

Q6 **An architect evaluating MCP for an enterprise deployment wants to understand how tool discovery works at runtime. Which statement is accurate?**

A Tool discovery in MCP requires the client to have every tool's schema hard-coded at build time, with no runtime discovery capability

B **MCP clients can query a connected MCP server at runtime to discover the tools/resources it exposes (including their schemas), allowing the available capabilities to be introduced or updated on the server side without requiring client redeployment**

C Tool discovery is not a concept supported by MCP; it only supports a single fixed tool per server

D MCP servers must discover the client's capabilities rather than the reverse

CORRECT ANSWER **B**

WHY A core design goal of MCP is runtime tool/resource discovery — clients can query a server to learn what capabilities it currently exposes, letting servers evolve or add tools without forcing every client to be rebuilt and redeployed in lockstep. Requiring everything hard-coded at build time defeats this purpose, MCP explicitly supports multiple tools/resources per server (not just one), and the discovery relationship runs client-queries-server for capabilities, not the reverse.

Q7 **A logistics company is deciding between exposing its warehouse management tools via MCP versus direct API calls baked into the agent's code. A key requirement is that three separate internal teams (dispatch, inventory, and customer service) each want to build their own Claude-powered tools using the same warehouse capabilities, with consistent authorization enforcement across all three. Which factor most strongly favors MCP here?**

A MCP always produces lower latency than direct API calls in every scenario

B **MCP's standardized server model lets the warehouse capabilities be defined, secured, and exposed once, consistently, for reuse across multiple independent client teams, rather than each team re-implementing (and potentially inconsistently securing) direct API integration logic**

C MCP eliminates the need for any authentication whatsoever

D Direct API integration is architecturally impossible when multiple teams are involved

CORRECT ANSWER **B**

WHY The stated requirement — multiple independent teams reusing the same capabilities with consistent authorization — is exactly the reuse/consistency problem MCP's standardized server model addresses, centralizing tool definition and security enforcement rather than letting each team reinvent it (and risk inconsistent, weaker security). MCP does not inherently guarantee lower latency than direct calls, it does not eliminate the need for authentication (quite the opposite — servers must still enforce it), and direct API integration remains technically possible with multiple teams, just less consistent and more duplicative.

Q8 **A manufacturing company's architecture review flags that its MCP server for equipment control tools accepts any client connection without verifying client identity or enforcing scoped permissions, relying solely on network-level firewall rules. What is the risk?**

A. No risk, since MCP inherently handles all authentication and authorization automatically with no server-side configuration needed

B. Any client that can reach the server on the network (e.g., a compromised internal host) could invoke potentially sensitive equipment-control tools, since authentication/authorization must be explicitly implemented at the MCP server layer and is not provided for free by the protocol alone

C. This is an acceptable long-term design because firewalls are sufficient for any equipment-control use case

D. MCP servers cannot expose write/control actions, only read-only data, so this concern does not apply

CORRECT ANSWER B

WHY MCP defines a protocol for tool discovery and invocation, but authentication and fine-grained authorization are responsibilities that must be explicitly designed and enforced by the server implementation; relying only on network-level controls leaves the server exposed to any client that reaches it on the network, including a compromised internal host. Assuming the protocol handles this "automatically" is a misunderstanding of MCP's scope, network firewalls alone are insufficient defense-in-depth for sensitive control actions, and MCP servers are fully capable of exposing write/control-capable tools, which makes proper authorization even more critical, not irrelevant.

Q9 **A telecom provider is building a system where a top-level "customer intent" agent must coordinate with several specialized agents (billing agent, technical-support agent, retention-offers agent), each potentially built on different underlying models or frameworks, passing structured task requests and receiving structured results. What protocol consideration is most relevant to evaluate?**

A **Whether an agent-to-agent (A2A) communication standard can be used to provide a consistent way for the orchestrator to delegate tasks and receive results across heterogeneous agents, regardless of each sub-agent's internal implementation**

B Whether all sub-agents can be forced to use the exact same underlying model to simplify communication

C Whether CLI piping between terminal windows is sufficient for this production customer-facing system

D Whether the agents can simply share a single combined system prompt instead of separate coordination

CORRECT ANSWER **A**

WHY A2A-style standards are intended to provide a consistent, framework/model-agnostic way for autonomous agents to delegate tasks and exchange structured results, which is exactly the heterogeneous multi-agent coordination challenge described. Requiring all sub-agents to use the same model defeats the purpose of specialized agents potentially built differently, CLI piping between terminal windows is a manual developer workflow, not a production coordination mechanism, and merging everything into a single shared system prompt would collapse the architecture back into a monolith, losing the benefits of specialized, independently operated agents.

Q10 **An architect is comparing MCP and direct API integration for a new internal project and is told by a colleague: "MCP is strictly better in every situation, so we should always use it." How should the architect evaluate this claim?**

A Agree completely; MCP should always be used regardless of context

B **Push back: protocol choice should be evaluated against the actual requirements — number of clients needing reuse, need for dynamic tool discovery, operational overhead tolerance, and team maturity with the protocol — since for a simple, single-client, stable integration, direct API calls may be simpler and sufficient**

C Disagree completely and conclude MCP should never be used under any circumstances

D Agree, but only because MCP guarantees lower cost than any other integration method

CORRECT ANSWER B

WHY Sound architectural judgment requires matching the protocol to actual needs (reuse across clients, discovery requirements, operational complexity tolerance) rather than applying a blanket rule; a simple, single-client, stable integration may not justify MCP's additional infrastructure and conceptual overhead. Blanket endorsement or blanket rejection both ignore legitimate trade-offs, and MCP does not come with a guarantee of lower cost — cost depends on the specific implementation and usage pattern.

Q11 **A government agency integrating Claude with a legacy mainframe system (accessible only via a decades-old command-line interface with no REST API available) needs the agent to retrieve case records. Which integration approach is most pragmatic given this constraint?**

A Refuse to integrate until the mainframe is replaced with a modern REST API

B **A CLI-based integration layer that wraps the mainframe's command-line interface, translating agent tool calls into appropriately sanitized and permission-checked CLI invocations, potentially exposed to the agent via a thin MCP or direct-API wrapper**

C Direct database access bypassing the mainframe's interface entirely, ignoring any access controls the mainframe enforces

D Abandon structured integration and have a human manually relay all information between the agent and the mainframe

CORRECT ANSWER B

WHY When the only available interface to a legacy system is a CLI, a CLI-wrapping integration layer (with proper sanitization and permission checks, potentially fronted by MCP or a direct API for the agent-facing side) is the pragmatic way to bridge old and new systems without an immediate system replacement. Refusing to integrate until a full modernization project completes may not be realistic on the agency's timeline, bypassing the mainframe's own access controls via direct database access is a serious security risk, and reverting to fully manual human relay defeats the purpose of building an automated integration at all.

Q12 **A SaaS company's security team is reviewing an MCP-based integration and wants to confirm that permission boundaries are correctly scoped. Which configuration represents a properly scoped permission boundary?**

A The MCP server exposes a single generic "execute_admin_command" tool that accepts arbitrary commands as a string parameter, trusting the client to only send appropriate ones

B **The MCP server exposes narrowly defined tools (e.g., `get_invoice_status`, `update_customer_address`) each mapped to specific, least-privilege backend permissions, so the server itself enforces what actions are possible regardless of what the client requests**

C The MCP client enforces all permission boundaries, while the server executes anything it receives without validation

D Permission boundaries are unnecessary once TLS encryption is enabled for the connection

CORRECT ANSWER **B**

WHY Properly scoped permission boundaries mean the server defines narrow, purpose-specific tools tied to least-privilege backend permissions, so the server itself — not client-side trust — enforces what is actually possible to do, which is the correct security posture for any protocol-based integration. A generic "execute arbitrary command" tool defeats the purpose of scoped permissions by trusting client-supplied strings, having the client be solely responsible for enforcement while the server blindly executes is backwards and dangerous, and TLS only protects data in transit — it does nothing to constrain what actions a connection is authorized to perform.

Q13 **An insurance company is deciding how a Claude-based underwriting agent should communicate with a third-party actuarial risk-modeling service operated by an external vendor, where the vendor's service is itself an independent reasoning system (not a simple deterministic API) that may need to ask clarifying questions back. Which protocol characteristic becomes most important to evaluate?**

A Whether the connection supports a simple one-shot request/response pattern only, since that is always sufficient for any integration

B **Whether the protocol supports richer, potentially multi-turn or stateful interaction patterns appropriate for coordinating with another reasoning agent that may need back-and-forth clarification, rather than assuming a single stateless call will suffice — a consideration central to A2A-style designs**

C Whether the vendor's service can be made to run entirely inside Claude's own infrastructure

D Whether the integration can skip authentication since the vendor is "trusted" as a business partner

CORRECT ANSWER **B**

WHY When the remote party is itself an autonomous reasoning system that may need clarification or multi-turn exchange, the protocol must support richer interaction patterns than a single stateless request/response, which is a central consideration in agent-to-agent style protocol design. Assuming one-shot request/response always suffices ignores the stated need for back-and-forth clarification, it's not feasible or appropriate to force a third-party vendor's service to run inside Claude's own infrastructure, and treating a business partnership as a substitute for proper authentication is a security anti-pattern regardless of trust level.

Q14 **A healthcare network's architecture team is choosing between exposing an MCP server for its internal lab-results system versus building direct point-to-point API integrations for each of its four separate Claude-powered applications (patient portal, physician dashboard, billing assistant, research tool). What long-term maintenance consideration favors MCP in this case?**

A MCP guarantees that lab-results data will always be more accurate than data fetched via direct API calls

B **With four separate applications needing the same underlying capability, a single well-governed MCP server centralizes tool definitions, schema changes, and authorization logic in one place, whereas four separate point-to-point integrations would each need to be updated independently whenever the lab-results system's API changes**

C MCP removes the need for any ongoing maintenance once deployed

D Direct API integration is objectively impossible when there is more than one client application

CORRECT ANSWER **B**

WHY With multiple independent client applications needing the same backend capability, centralizing the integration logic, schema, and authorization in one MCP server avoids the maintenance burden and drift risk of updating four separate point-to-point integrations whenever the underlying system changes — a classic "write once, reuse many" argument for standardized protocols. MCP has no bearing on data accuracy itself, no integration approach eliminates all ongoing maintenance, and direct API integration remains technically possible with multiple clients, just less efficient to maintain at scale.

Q15 **An architect must decide whether a new internal "meeting notes summarizer" tool, used only by a single lightweight internal Slack bot with no plans for reuse elsewhere, should be built as a full MCP server or a simple direct API call from the bot's backend. The team has limited time before a deadline. What is the most defensible recommendation?**

A Build a full MCP server regardless of time constraints, since MCP should be used for literally every integration without exception

B **Given the narrow scope (single client, no anticipated reuse, time pressure), a direct API integration is a reasonable and pragmatic choice, with the option to refactor into an MCP server later if reuse needs emerge**

C Skip building any integration and have the bot operator manually copy-paste meeting notes into Claude's chat interface

D Use agent-to-agent protocol negotiation, since any tool call between two pieces of software counts as agent-to-agent communication

CORRECT ANSWER **B**

WHY Matching protocol complexity to actual current needs — a single client, no reuse requirement, and a deadline — favors the simpler direct API integration, with the architecturally sound fallback of refactoring to MCP later if the tool's scope genuinely expands to multiple clients. Mandating full MCP infrastructure for every integration regardless of scope or timeline ignores real engineering trade-offs, abandoning automation for manual copy-paste defeats the purpose of building the tool at all, and labeling a simple tool call as "agent-to-agent communication" misapplies a term meant for coordination between autonomous reasoning agents, not a basic utility function call.

Q16 **An architect must decide whether a new capability — "generate a weekly sales summary using the company's documented five-step formatting and approval procedure" — should be built as an MCP server or packaged as a Skill (reusable instructions/know-how Claude loads when relevant). The procedure requires no live call to any external system once the raw sales figures are already in context. What is the deciding question, and what is the right answer here?**

A The deciding question is how many steps the procedure has; five or more steps always requires MCP

B **The deciding question is whether the capability must reach out and talk to another system at the moment it runs — since this procedure fetches nothing live and is purely know-how applied to data already in context, it is Skill-shaped, not MCP-shaped**

C The deciding question is irrelevant, since MCP and Skills are two names for the same mechanism

D The deciding question is which team owns the budget for the integration, since that always determines the protocol choice

CORRECT ANSWER B

WHY The discriminating test between MCP and a Skill is whether the capability needs a live call to another system at execution time: MCP is for capabilities that must fetch or act on something external while they run, while a Skill packages know-how or a procedure that operates on data already available and needs nothing fetched — this formatting/approval procedure is exactly the latter. Step count has no bearing on the choice, MCP and Skills are distinct mechanisms satisfying different needs (and are frequently confused as distractors for this reason), and budget ownership is an organizational detail, not an architectural discriminator between the two.

Q17 **A hospital's IT governance team discovers that, by default, any engineer running a Claude-based coding assistant internally can connect any MCP server they find or build, with no central review of what that server does before it connects to internal clinical systems. Anthropic's own documentation notes it reviews community connectors only against directory-listing criteria and does not security-audit or manage third-party MCP servers. What should the governance team do?**

A Accept the default as sufficient, since Anthropic's directory review functions as a full security audit of every connectable server

B **Apply the administrative controls documented for this exact scenario: choose from disabling MCP servers entirely, deploying a fixed managed set engineers cannot add to, publishing an approved catalogue, or deliberately leaving engineers unrestricted only where that risk is acceptable — rather than relying on an assumed audit that does not exist**

C Ban all MCP usage permanently across the organization with no further evaluation, since no middle ground exists

D Delegate the decision entirely to each individual engineer, since server selection is a personal productivity choice with no organizational risk

CORRECT ANSWER **B**

WHY Because any user can connect any MCP server by default and Anthropic's listing review is not a security audit of what a server actually does, an organization with internal clinical systems at stake needs to deliberately choose among the documented administrative controls — full restriction, a fixed managed set, an approved catalogue, or informed unrestricted access — rather than assume protection that isn't there. Treating directory listing as a security audit is a factual misunderstanding of what Anthropic actually reviews, a blanket permanent ban ignores that lighter-weight governed options exist and are documented for exactly this situation, and leaving the decision entirely to individual engineers abandons organizational responsibility for a risk that clearly has organization-wide consequences.

Q18 **A vendor's MCP client application is found to retain and act on the full contents of every other MCP server's conversation turns within the same session, using that cross-server context to decide what to send to a billing server. A reviewer flags this against the protocol's architecture. What principle is being violated?**

A Nothing is violated; MCP has no stated position on what a server can see of the conversation

B **A core MCP design principle: the host, not individual servers, controls connection permissions and consent, and holds the full conversation history — servers should not be able to read the whole conversation or see into other servers, so cross-server visibility of this kind runs counter to the protocol's intended isolation**

C This is required behavior, since MCP mandates that all connected servers share full visibility into each other's data for consistency

D This only matters if the servers are hosted by different vendors; same-vendor servers are exempt from the isolation principle

CORRECT ANSWER **B**

WHY MCP's architecture places the host in control of connection permissions, security policy, and consent, with full conversation history staying at the host level specifically so that individual servers are not able to read the whole conversation or see into other servers — a design principle meant to contain the blast radius of any single server. Claiming MCP has no position on this misstates the documented architecture, the protocol does not mandate shared cross-server visibility (it does the opposite), and the isolation principle is about server boundaries generally, not contingent on whether servers happen to share a vendor. --